Reading: inside the GPU

Short notes for session 12, based on chapter 4 of *Programming Massively Parallel Processors*. The slides (`slides.html`) animate every idea below.

---

1. The machine

- A GPU is an array of **streaming multiprocessors (SMs)**. Each SM has many **streaming processors** (arithmetic lanes), shared control logic and fast on-chip memory.
- **Global memory** is off-chip DRAM: gigabytes, high bandwidth, but hundreds of cycles away.
- Example (H100): 132 SMs × 128 processors = 16,896 processors.

---

2. Blocks go to SMs whole

- Each block runs entirely on **one SM**, with all its threads there at the same time. An SM can hold several blocks; the rest of the grid waits for a free slot.
- That is why threads of a block can cooperate: a barrier (`__syncthreads()`) and shared memory.
- Blocks **cannot wait for each other**. In return, the hardware may run them in any order and any number at a time: the same program runs on a small or a large GPU (**transparent scalability**).

Barrier rule: a `__syncthreads()` must be reached by **all** threads of the block or by none. Each call site is a different barrier, so never put one inside an `if` that depends on `threadIdx`.

---

3. Warps

- Inside the SM, a block is cut into **warps of 32 consecutive threads**: warp w = threads $32w$ … $32w + 31$.
- 2D/3D blocks are first flattened row by row: `linear = y * blockDim.x + x`.
- A warp executes **one instruction at a time for all its threads** (SIMD hardware, written as scalar threads: SIMT).
- Warps per block = ⌈threads per block / 32⌉. A block of 48 threads uses 2 warps; the second has 16 idle lanes. Prefer block sizes that are multiples of 32.

---

4. Control divergence

- If threads of the **same warp** take different branches, the warp runs each path in turn while the other lanes wait.
- A condition can diverge only if it differs **inside a warp** (it depends on `threadIdx` or on data). `if (blockIdx.x == 0)` never diverges.
- Boundary checks (`if (i < n)`) diverge in at most one warp per edge; the cost shrinks as the data grows.
- After a branch, do not assume the warp is back in step. Use `__syncwarp()` if needed.

---

5. Latency tolerance

- An SM holds many more warps than it can run at once (H100: 2048 threads for 128 processors).

- When a warp waits for memory, the SM switches to a ready warp **at no cost**: every warp keeps its registers in the SM. With enough warps, memory latency is hidden.

6. Occupancy

```
occupancy = warps resident on the SM / maximum warps of the SM

resident blocks = min( threads per SM / threads per block,
                       block slots per SM,
                       registers per SM / (threads per block × registers per thread) )
```

With H100 limits (2048 threads, 32 blocks, 65,536 registers per SM):

Kernel	Resident	Occupancy	Limit
256 threads/block, 30 registers	8 blocks	100 %	—
32 threads/block	32 blocks = 1024 threads	50 %	block slots
768 threads/block	2 blocks = 1536 threads	75 %	threads
512 threads/block, 33 registers	3 blocks = 1536 threads	75 %	registers

Full occupancy on an H100 needs at most **32 registers per thread** (65,536 / 2048). Query your own GPU with `cudaGetDeviceProperties` (`multiProcessorCount`, `maxThreadsPerBlock`, `maxThreadsPerMultiProcessor`, `regsPerBlock`, `warpSize`).

Check yourself

1. A kernel uses 128 threads per block and a grid of 16 blocks. How many warps are launched?
2. Which of these can diverge: `if (threadIdx.x < 16)`, `if (blockIdx.y > 3)`, `if (data[i] > 0)`?
3. Why is it wrong to call `__syncthreads()` only in the `else` branch of `if (threadIdx.x % 2 == 0)`?
4. A kernel with 256 threads per block uses 40 registers per thread. What is its occupancy on an H100, and what limits it?

> **Answers**
>
> 1. 128 / 32 = 4 warps per block × 16 = **64 warps**.
> 2. `threadIdx.x < 16` (lanes 0–15 vs 16–31 of the first warp) and `data[i] > 0` (depends on the data). `blockIdx.y > 3` is the same for the whole warp.
> 3. The even threads never reach that barrier, so the odd threads wait forever (or the result is undefined).
> 4. 65,536 / (256 × 40) = 6.4 → **6 blocks** = 1536 threads = **75 %**, limited by registers.